

COMPILER DESIGN LAB 7&8

Date @March 17, 2026

Name: Devadathan N R
Reg No: 230905010
Class: CSE - A1
Roll No: 04

LAB 7: RD PARSER FOR DECLARATION STATEMENTS

LAB 8: RD PARSER FOR ARRAY DECLARATIONS AND EXPRESSION STATEMENTS

Question

Design the recursive descent parser to parse array declarations and expression statements with error reporting.

GRAMMAR:

```
Program      → main ( ) { Declarations StatementList }
Declarations → DataType IdentifierList ; Declarations | ε
DataType     → int | char
IdentifierList → id
             | IdentifierList , id
             | id [ number ]
             | IdentifierList , id [ number ]
StatementList → Statement StatementList | ε
Statement     → AssignStat ;
             | DecisionStat
AssignStat    → id = Expn
DecisionStat  → if ( Expn ) { StatementList } Dprime
Dprime        → else { StatementList } | ε
Expn          → SimpleExpn Eprime
Eprime        → Relop SimpleExpn | ε
SimpleExpn    → Term Seprime
Seprime       → Addop Term Seprime | ε
Term          → Factor Tprime
Tprime        → Mulop Factor Tprime | ε
Factor        → id | num
Relop         → == | != | <= | >= | < | >
Addop         → + | -
Mulop         → * | / | %
```

CODE:

lexicalAnalyser.c

```
#include "la.h"
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define BUFFER_SIZE 4096

static char buffer[BUFFER_SIZE];
static int bufSize = 0;
static int LB = 0, FP = 0;
```

```

static int row = 1, col = 1;
static int initialized = 0;

const char *keywords[] = {
    "int", "float", "double", "char", "void", "long", "short", "signed", "unsigned",
    "if", "else", "while", "for", "do", "switch", "case", "default",
    "break", "continue", "return", "goto",
    "auto", "register", "static", "extern", "const", "volatile",
    "struct", "union", "enum", "typedef", "sizeof",
    "printf", "scanf", "malloc", "free", "strlen", "strcpy", "strcmp", "exit",
    "main"
};

int keywordCount = sizeof(keywords)/sizeof(keywords[0]);

void loadBuffer(FILE *fp){
    bufSize = fread(buffer, 1, BUFFER_SIZE-1, fp);
    buffer[bufSize] = '\0';
}

char advanceChar(){
    char c = buffer[FP++];
    if(c=='\n'){ row++; col=1; }
    else col++;
    return c;
}

char peekChar(){ return buffer[FP]; }

int isKeyword(char *str){
    for(int i=0; i<keywordCount; i++)
        if(strcmp(str, keywords[i])==0) return 1;
    return 0;
}

int isSpecialSymbol(char c){
    return (c=='(' || c==')' || c=='{' || c=='}' || c=='[' || c==']' || c==';' || c==',' );
}

void skipWhiteSpace(){
    while(peekChar()==' ' || peekChar()=='\t' || peekChar()=='\n')
        advanceChar();
}

void skipSingleLineComment(){
    while(advanceChar()!='\n');
}

void skipMultiLineComment(){
    char c;
    while(FP<bufSize){
        c=advanceChar();
        if(c=='*' && peekChar()=='/'){
            advanceChar();
            break;
        }
    }
}

void skipPreprocessor(){

```

```

    while(advanceChar()!='\n');
}

struct token getNextToken(FILE *fp)
{
    struct token tk;

    if(!initialized){
        loadBuffer(fp);
        initialized=1;
    }

start:
    skipWhiteSpace();

    tk.row=row;
    tk.col=col;

    LB=FP;
    char c=advanceChar();

    if(c=='\0'){
        strcpy(tk.type,"EOF");
        return tk;
    }

    if(c=='#'){
        skipPreprocessor();
        goto start;
    }

    if(c=='/'){
        if(peekChar()=='/'){
            advanceChar();
            skipSingleLineComment();
            goto start;
        }
        if(peekChar()=='*'){
            advanceChar();
            skipMultiLineComment();
            goto start;
        }
    }

    /* string literal */
    if(c==''){
        while(peekChar()!=''' && peekChar()!='\0') advanceChar();
        advanceChar();

        int len=FP-LB;
        strncpy(tk.lexeme,buffer+LB,len);
        tk.lexeme[len]='\0';
        strcpy(tk.type,"STRING_LITERAL");
        return tk;
    }

    /* identifier or keyword */
    if(isalpha(c)||c=='_'){
        while(isalnum(peekChar())||peekChar()=='_')
            advanceChar();
    }
}

```

```

        int len=FP-LB;
        strncpy(tk.lexeme,buffer+LB,len);
        tk.lexeme[len]='\0';

        if(isKeyword(tk.lexeme))
            strcpy(tk.type,"KEYWORD");
        else
            strcpy(tk.type,"Identifier");

        return tk;
    }

/* number */
if(isdigit(c)){
    while(isdigit(peekChar())) advanceChar();

    int len=FP-LB;
    strncpy(tk.lexeme,buffer+LB,len);
    tk.lexeme[len]='\0';

    strcpy(tk.type,"Number");
    return tk;
}

/* operators */

/* double char operators */
if(c=='=' && peekChar()=='='){
    advanceChar();
    strcpy(tk.lexeme,"==");
    strcpy(tk.type,"SYMBOL");
    return tk;
}
if(c=='!' && peekChar()=='='){
    advanceChar();
    strcpy(tk.lexeme,"!=");
    strcpy(tk.type,"SYMBOL");
    return tk;
}
if(c=='<' && peekChar()=='='){
    advanceChar();
    strcpy(tk.lexeme,"<=");
    strcpy(tk.type,"SYMBOL");
    return tk;
}
if(c=='>' && peekChar()=='='){
    advanceChar();
    strcpy(tk.lexeme,">=");
    strcpy(tk.type,"SYMBOL");
    return tk;
}

/* single char operators */
if(c=='+' || c=='-' || c=='*' || c=='/' || c=='%' ||
    c=='=' || c=='<' || c=='>'){
    tk.lexeme[0]=c;
    tk.lexeme[1]='\0';
    strcpy(tk.type,"SYMBOL");
    return tk;
}

```

```

    }

    /* special symbol */
    if(isSpecialSymbol(c)){
        tk.lexeme[0]=c;
        tk.lexeme[1]='\0';
        strcpy(tk.type,"SYMBOL");
        return tk;
    }

    /* unknown */
    tk.lexeme[0]=c;
    tk.lexeme[1]='\0';
    strcpy(tk.type,"UNKNOWN");

    return tk;
}

```

la.h

```

#ifndef LA_H
#define LA_H

#include <stdio.h>
#define MAX_LEXEME 100

struct token {
    char lexeme[MAX_LEXEME];
    char type[20];
    int row;
    int col;
};

struct token getNextToken(FILE *fp);

#endif

```

rd.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "la.h"

FILE *fp;
struct token tkn;

void program();
void declarations();
void data_type();
void identifier_list();
void statement_list();
void statement();
void assign_stat();
void decision_stat();
void expn();
void eprime();
void simple_expn();

```

```

void seprime();
void term();
void tprime();
void factor();
void dprime();
void relop();
void addop();
void mulop();

void syntaxError(char *expected){
    printf("Syntax error at row %d, col %d: expected %s but found '%s'\n", tkn.row, tkn.col, expected, tk
n.lexeme);
    exit(1);
}

void match(char *expected)
{
    if (strcmp(expected, tkn.lexeme) == 0){tkn = getNextToken(fp);}
    else{syntaxError(expected);}
}

void program()
{
    match("main");
    match("(");
    match(")");
    match("{");
    declarations();
    statement_list();
    match("}");
    printf("===== Parsing successful =====\n");
}

void declarations()
{
    if (strcmp(tkn.lexeme, "int") == 0 || strcmp(tkn.lexeme, "char") == 0)
    {
        data_type();
        identifier_list();
        match(";");
        declarations();
    }
}

void data_type()
{
    if (strcmp(tkn.lexeme, "int") == 0 || strcmp(tkn.lexeme, "char") == 0)
    {tkn = getNextToken(fp);}
    else
    {syntaxError("int or char");}
}

void identifier_list()
{
    if (strcmp(tkn.type, "Identifier") == 0)
    {
        tkn = getNextToken(fp);
        if (strcmp(tkn.lexeme, "[") == 0)
        {
            match("[");

```

```

        if (strcmp(tkn.type, "Number") == 0)
        {
            tkn = getNextToken(fp);
            match("]");
        }
        else
        {
            syntaxError("Number");
        }
    }
    if (strcmp(tkn.lexeme, ",") == 0)
    {
        match(",");
        identifier_list();
    }
}
else
{
    syntaxError("Identifier");
}
}

void statement_list()
{
    if (strcmp(tkn.lexeme, "if") == 0 || strcmp(tkn.type, "Identifier") == 0)
    {
        statement();
        statement_list();
    }
}

void statement()
{
    if (strcmp(tkn.lexeme, "if") == 0)
        decision_stat();
    else if (strcmp(tkn.type, "Identifier") == 0)
        assign_stat();
    else
        syntaxError("if or Identifier");
}

void assign_stat()
{
    if (strcmp(tkn.type, "Identifier") == 0)
    {
        tkn = getNextToken(fp);
        match("=");
        expn();
        match(";");
    }
    else
    {
        syntaxError("Identifier");
    }
}

void expn()
{
    simple_expn();
    eprime();
}

```

```

}

void eprime()
{
    if (strcmp(tkn.lexeme, "==") == 0 || strcmp(tkn.lexeme, "!=") == 0 ||
        strcmp(tkn.lexeme, "<=") == 0 || strcmp(tkn.lexeme, ">=") == 0 ||
        strcmp(tkn.lexeme, "<") == 0 || strcmp(tkn.lexeme, ">") == 0)
    {
        relop();
        simple_expn();
    }
    // epsilon case handled implicitly
}

void simple_expn()
{
    term();
    seprime();
}

void seprime()
{
    if (strcmp(tkn.lexeme, "+") == 0 || strcmp(tkn.lexeme, "-") == 0)
    {
        addop();
        term();
        seprime();
    }
    // epsilon case handled implicitly
}

void term()
{
    factor();
    tprime();
}

void tprime()
{
    if (strcmp(tkn.lexeme, "*") == 0 || strcmp(tkn.lexeme, "/") == 0 || strcmp(tkn.lexeme, "%") == 0)
    {
        mulop();
        factor();
        tprime();
    }
    // epsilon case handled implicitly
}

void factor()
{
    if (strcmp(tkn.type, "Identifier") == 0 || strcmp(tkn.type, "Number") == 0)
    {
        tkn = getNextToken(fp);
    }
    else
    {
        syntaxError("Identifier or Number");
    }
}

```



```

void decision_stat()
{
    match("if");
    match("(");
    expn();
    match(")");
    match("{");
    statement_list();
    match("}");
    dprime();
}

void dprime()
{
    if (strcmp(tkn.lexeme, "else") == 0)
    {
        match("else");
        match("{");
        statement_list();
        match("}");
    }
    // epsilon handled implicitly
}

void relop()
{
    if (strcmp(tkn.lexeme, "==") == 0 || strcmp(tkn.lexeme, "!=") == 0 ||
        strcmp(tkn.lexeme, "<=") == 0 || strcmp(tkn.lexeme, ">=") == 0 ||
        strcmp(tkn.lexeme, "<") == 0 || strcmp(tkn.lexeme, ">") == 0)
    {
        tkn = getNextToken(fp);
    }
    else
    {
        syntaxError("Relational Operator");
    }
}

void addop()
{
    if (strcmp(tkn.lexeme, "+") == 0 || strcmp(tkn.lexeme, "-") == 0)
    {
        tkn = getNextToken(fp);
    }
    else
    {
        syntaxError("+ or -");
    }
}

void mulop()
{
    if (strcmp(tkn.lexeme, "*") == 0 || strcmp(tkn.lexeme, "/") == 0 || strcmp(tkn.lexeme, "%") == 0)
    {
        tkn = getNextToken(fp);
    }
    else
    {
        syntaxError("* or / or %");
    }
}

```

```

}

int main()
{
    fp = fopen("demo.c", "r");
    if (!fp)
    {
        printf("Error opening input file\n");
        return 1;
    }
    tkn = getNextToken(fp);
    program();
    return 0;
}

```

demo.c

Error Case:(3 errors detected one by one)

```

main() {
    int a, b[10]
    a = 5
    if (a < 10) {
        a = a + ;
    }
}

```

Correct Case:

```

main() {
    int a, b[10];
    a = 5;
    if (a < 10) {
        a = a + 1;
    }
}

```

OUTPUT:

Error Case:

```

CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/08_LAB$ gcc rd.c lexicalAnalyser.c -o parser
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/08_LAB$ ./parser
Syntax error at row 3, col 5: expected ; but found 'a'
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/08_LAB$ ./parser
Syntax error at row 4, col 5: expected ; but found 'if'
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/08_LAB$ ./parser
Syntax error at row 5, col 17: expected Identifier or Number but found ';'

```

Correct Case:

```

CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/08_LAB$ gcc rd.c lexicalAnalyser.c -o parser
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/08_LAB$ ./parser
===== Parsing successful =====
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/08_LAB$ █

```